Document operating system for RAG

Local-first dynamic knowledge base for documents, RAG, Obsidian, Graphify, and agent memory

2026-07-08

Contents

Executive summary

This plan designs a local-first **Document operating system for RAG**. The purpose is to ingest books, magazines, PDFs, scanned documents, university material, RSS feeds, web pages, screenshots, and other human-readable sources, then turn them into a precise, searchable, source-linked knowledge base.

The target is not a simple vector database. A vector database is only one storage layer. The correct target is a controlled document pipeline that keeps raw evidence, extracted text, Markdown, JSON, page images, tables, figures, chunks, embeddings, full-text indexes, graph entities, Obsidian notes, and reviewed memory entries as separate layers.

The system should work like this:

```
source material
  -> source registry
  -> document router
  -> extraction pipeline
  -> selective OCR / vision validation
  -> quality scoring
  -> structured Markdown and JSON
  -> chunking with page provenance
  -> hybrid search index
```

```
-> RAG answer service with citations
-> reviewed Obsidian / Graphify / memory promotion
-> update loop for changing sources
```

The main rule is: **never lose provenance**. Every answer produced from the knowledge base should be traceable back to a source file, page, chunk, extraction method, timestamp, and confidence score.

Corrected hardware assumptions

The active processing machine is one AI-PC with:

- one active GPU: **RTX 5060 Ti 16 GB**
- CPU: **Intel i7-10750 class**
- RAM: **32 GB DDR4**
- laptops: endpoints only for viewing, uploading, checking, and controlling jobs

Do not design for unused GPUs. Do not assume multi-GPU scheduling. The system must run with one GPU worker queue and one CPU/disk extraction queue.

Minimal hardware spec

This is the minimum practical specification for the first usable version:

- 6-core / 12-thread CPU or better
- 32 GB RAM
- NVIDIA GPU with 12 GB VRAM or better
- 1 TB SSD for working artefacts
- separate large storage for raw documents and generated outputs
- Linux host with Docker, Python, and stable NVIDIA drivers

Your current AI-PC fits this class, mainly because of the 16 GB VRAM GPU. The 32 GB RAM is workable, but batching must be controlled.

Recommended hardware spec

This is the recommended target if the system becomes large:

- 8 to 12 CPU cores or better
- 64 GB RAM
- NVIDIA GPU with 16 GB VRAM or better
- 2 TB NVMe for active processing
- 4 TB+ SSD/HDD for raw library, page images, and artefact archives
- daily backup target
- UPS if running long OCR/index jobs

The first upgrade I would consider is **RAM to 64 GB**, not another GPU. Large PDFs, page rendering, OCR workers, embedding batches, Qdrant, and dashboard services can pressure 32 GB before they fully saturate the GPU.

Design principles

1. Router first, OCR second

Do not OCR every PDF. That is slow and can damage precision when the embedded text is already good.

Every file should first go through a document router that decides:

- is it a digital PDF?
- is it a scanned PDF?
- does it contain tables?
- does it contain formulas?

- is it multi-column?
- does it contain charts, figures, or screenshots?
- is it Romanian, English, or mixed?
- is it a university file, book, article, RSS item, or web capture?
- does it require a fallback tool?

Then the system chooses the cheapest reliable extraction path.

2. Store layers, not just chunks

The system should not only store vector chunks. It should preserve:

- raw file
- source metadata
- extracted plain text
- extracted Markdown
- page images
- OCR output
- layout JSON
- table JSON / CSV
- figure captions
- chunks
- embeddings
- full-text search rows
- graph nodes and edges
- Obsidian notes
- memory promotion decisions

Vectors are disposable. Raw sources and structured extraction artefacts are not.

3. Hybrid retrieval, not vector-only retrieval

Vector search is useful for semantic similarity, but it is weak with exact facts, module codes, legal wording, command names, page references, table labels, and uncommon Romanian or technical terms.

Use hybrid retrieval:

```
query
  -> keyword / BM25 / SQLite FTS
  -> dense vector search
  -> optional sparse vector search
  -> metadata filters
  -> reranker
  -> answer with source citations
```

4. Memory promotion must be gated

Do not automatically push every extracted sentence into Obsidian, Graphify, or long-term agent memory.

Use promotion levels:

- L0 raw evidence
- L1 extracted document artefacts
- L2 indexed chunks
- L3 curated Obsidian notes
- L4 Graphify concepts and relationships
- L5 stable agent memory

Only L3 to L5 should be treated as long-term knowledge. Everything else is evidence and retrieval material.

5. Precision beats automation speed

The system should prefer slower reliable processing over fast broken extraction. Every extraction pass must produce a confidence score and a failure report.

Target architecture

Main services

Use a small number of clear services:

- **ingest-api**: receives files, URLs, RSS subscriptions, and manual jobs
- **registry-db**: SQLite database for source records, documents, pages, jobs, chunks, and quality scores
- **worker-extract**: CPU-heavy parser for digital documents
- **worker-ocr-vlm**: GPU-gated OCR and vision worker
- **worker-embed**: embedding worker
- **qdrant**: vector database
- **fts-index**: SQLite FTS5 full-text index
- **rag-api**: retrieval and answer service
- **exporter-obsidian**: Markdown note generator
- **exporter-graphify**: graph node and edge generator
- **review-ui**: low-confidence and promotion review queue
- **scheduler**: systemd timer jobs for RSS, changed files, and re-indexing

Folder layout

```
/srv/ai-workdesk/kb/
  raw/
    books/
    magazines/
    university/
    rss/
    web/
    images/
  registry/
    kb.sqlite
  artefacts/
    extracted_text/
    extracted_markdown/
    page_images/
    ocr_json/
    layout_json/
    tables/
    figures/
    chunks/
    reports/
  indexes/
    qdrant/
    sqlite_fts/
  exports/
    obsidian/
    graphify/
    memory_candidates/
  eval/
    golden_docs/
    extraction_scores/
    retrieval_scores/
```

Job state machine

Every source should move through explicit states:

```
REGISTERED
  -> ROUTED
  -> EXTRACTED
  -> VALIDATED
  -> CHUNKED
  -> EMBEDDED
  -> INDEXED
  -> REVIEWED
  -> PROMOTED
  -> AVAILABLE_FOR_RAG
```

Failed jobs should not disappear. They should move to:

```
NEEDS_RETRY
NEEDS_SECONDARY_OCR
NEEDS_MANUAL_REVIEW
BLOCKED_UNSUPPORTED_FORMAT
```

Source registry

The registry is the control plane. It prevents duplicate work and allows reliable updates.

Recommended tables:

sources

```
source_id
source_kind: file | rss | url | folder_watch | manual_note
original_location
canonical_location
sha256
size_bytes
created_at
registered_at
last_checked_at
licence_status
privacy_class
status
```

documents

```
doc_id
source_id
title
author
language
content_type
page_count
parser_selected
ocr_required
created_at
updated_at
```

pages

```
page_id
doc_id
page_number
```

```
page_hash_text
page_hash_image
text_confidence
layout_confidence
table_confidence
ocr_used
needs_review
```

chunks

```
chunk_id
doc_id
page_start
page_end
section_title
chunk_type
text
summary
keywords
entities
embedding_model
source_hash
created_at
```

jobs

```
job_id
source_id
doc_id
job_type
state
priority
started_at
finished_at
error_message
retry_count
```

Document routing

The router should choose the extraction strategy before any expensive processing.

Routing rules

Condition	First action	Fallback
Digital PDF with good embedded text	PyMuPDF / pdfplumber / Docling	Marker or MinerU
Scanned PDF	OCR pipeline	VLM page validation
Technical PDF with formulas	MinerU / Marker	VLM validation for failed pages
Magazine or complex layout	Docling / Marker	OCR/VLM on low-confidence pages
Image or screenshot	OCR + VLM caption	manual review if low confidence
RSS article	feed parser + content extraction	browser snapshot if page is dynamic
Web page	trafilatura/readability/browser capture	screenshot + OCR if blocked
DOCX/PPTX/XLSX	Docling / Marker / MinerU	LibreOffice conversion as fallback

Router output

```
{
  "doc_id": "...",
  "route": "digital_pdf_structured",
  "language": "en_ro_mixed",
  "parser_chain": ["docling", "marker_fallback"],
  "ocr_policy": "selective",
  "vlm_policy": "low_confidence_pages_only",
  "expected_risk": "tables_and_two_column_layout"
}
```

Extraction pipeline

Pass 1: normal extraction

For digital PDFs, start with normal extraction:

- embedded text
- headings
- page numbers
- tables
- figures
- captions
- links
- references
- reading order
- formulas where available

Candidate tools:

- PyMuPDF for fast text and page metadata
- pdfplumber for text and table inspection
- Docling for structured document conversion
- Marker for Markdown/JSON/chunk conversion
- MinerU for technical and scientific documents

Pass 2: selective OCR and vision

Run OCR only where needed:

- page has no embedded text
- embedded text is corrupted
- Romanian diacritics or English text is broken
- table extraction score is low
- formula extraction is weak
- page has screenshots, diagrams, charts, or handwritten content
- reading order looks wrong

Candidate tools/models:

- PaddleOCR / PaddleOCR-VL for multilingual OCR and document parsing
- olmOCR for clean natural reading order from PDF/image documents
- Qwen2.5-VL or Qwen3-VL class model for selected page validation

Because the AI-PC has one 16 GB GPU, all OCR/VLM jobs should use a **GPU lock**. Only one heavy GPU job should run at a time.

Pass 3: extraction validation

Each page gets a score:

```
{
  "page_number": 12,
```

```
  "text_confidence": 0.94,
  "layout_confidence": 0.88,
  "table_confidence": 0.72,
  "ocr_used": true,
  "needs_review": false,
  "warnings": ["possible_table_loss"]
}
```

Validation checks:

- empty or near-empty page detection
- repeated header/footer removal
- wrong reading order detection
- broken words across line wraps
- table row/column count sanity
- formula preservation check
- image/figure presence check
- language mismatch check
- duplicate page detection

OCR and VLM strategy on one GPU

The 16 GB GPU is enough for a strong local-first pipeline, but only if the jobs are queued.

Recommended rule:

```
CPU extraction can run in parallel.
GPU OCR/VLM runs one job at a time.
Embedding batches run when OCR/VLM is idle.
The chat/retrieval model gets priority over background processing.
```

Practical scheduling

Use three queues:

- **fast queue**: small digital PDFs, RSS, webpages
- **batch queue**: books, magazines, long university packs
- **heavy queue**: scanned PDFs, OCR, VLM validation, table repair

Example policy:

```
08:00-22:00: light ingestion and retrieval priority
22:00-07:00: batch OCR, embeddings, index rebuilds
```

If the system is used during study or coding, pause heavy OCR.

Model and tool strategy

Local reasoning and agent control

Recommended candidates:

- Qwen3 14B quantized through Ollama for main local reasoning
- Qwen3 8B for faster classification and routing
- Qwen2.5-Coder 14B or similar for code repair and pipeline scripts

Use the main LLM for decisions, not raw extraction. The parser/OCR tools should do extraction. The LLM should decide routing, validate summaries, create metadata, and promote memory candidates.

Embeddings

Primary recommendation:

- **BGE-M3** for multilingual retrieval, especially English/Romanian mixed material

Why:

- multilingual
- dense retrieval
- sparse retrieval
- multi-vector retrieval
- supports longer input granularity than many small embedding models

Future candidate:

- **Qwen3 Embedding 0.6B / 4B / 8B**, tested after the first pipeline works

Do not change embedding models casually. Store the embedding model name and version per chunk.

Reranking

Add reranking after basic hybrid retrieval works.

Candidate rerankers:

- BGE reranker family
- Qwen3 reranker family

Reranking should be optional because it adds latency. Use it for final answers, not for every exploratory search.

Vision/document models

Candidate tools/models:

- PaddleOCR / PaddleOCR-VL for OCR and multilingual document parsing
- olmOCR for PDF/image documents where reading order matters
- Qwen2.5-VL or Qwen3-VL class model for low-confidence visual review

Do not ask a VLM to process entire books page by page unless the normal parser failed. VLMs are expensive and slower.

Chunking strategy

Chunking should preserve meaning and page evidence.

Chunk types

Use different chunk types:

- definition
- explanation
- procedure
- table
- figure
- code
- quote
- formula
- summary
- reference
- question-answer
- assignment/coursework note

Chunk size rules

Source type	Chunking rule
University notes	heading-based, 400-900 words

Source type	Chunking rule
Books	section-based, 700-1200 words
Magazines	article/section chunks
Tables	one table = one chunk plus summary
Code/commands	keep intact
Diagrams	OCR text + caption + linked page image
RSS/web	article sections with canonical URL
Definitions	separate atomic chunks

Parent-child structure

Use a tree:

```
document
  chapter
    section
      chunk
        table
        figure
        quote
```

This allows the RAG system to retrieve a small chunk but expand to the parent section when more context is needed.

Retrieval and answer service

Retrieval pipeline

```
user query
  -> query language detection
  -> query expansion / normalisation
  -> metadata filters
  -> SQLite FTS / BM25 search
  -> dense vector search
  -> optional sparse vector search
  -> result fusion
  -> reranking
  -> context pack
  -> answer with citations
```

Metadata filters

Support filters such as:

- source type: university, book, RSS, web, image
- module code
- author
- year
- language
- confidence threshold
- date ingested
- reviewed only
- include or exclude OCR-only pages
- private/public classification

Answer rule

The answer service should never say something came from the KB unless it can cite the source.

Every generated answer should include:

```
Sources:
- document title
- page number or article URL
- chunk ID
- extraction confidence
```

If retrieval is weak, it should say:

```
I found related material, but I cannot confirm the answer from the indexed sources.
```

Obsidian and Graphify export

Obsidian export

Obsidian should receive curated notes, not raw dumps.

Recommended note types:

- source notes
- concept notes
- module notes
- topic maps
- definition notes
- assignment support notes
- weekly university summaries
- reading summaries
- unresolved questions

Example frontmatter:

```yaml
title: OAuth 2.0 Authorization Code Flow
type: concept
source_count: 3
confidence: reviewed
created: 2026-07-08
updated: 2026-07-08
tags:
  - cybersecurity
  - authentication
  - university
```

Example note body:

```
# OAuth 2.0 Authorization Code Flow

## Summary
Short human-readable summary.

## Key points
- Point 1
- Point 2

## Source evidence
- COM4010 Lecture 04, page 22, chunk abc123
- Security textbook, page 118, chunk def456

## Related
- [[OpenID Connect]]
- [[Access Token]]
- [[Refresh Token]]
```

Graphify export

Graphify should receive entities and relationships:

```
{
  "node_type": "concept",
  "name": "OAuth 2.0 Authorization Code Flow",
  "aliases": ["auth code flow"],
  "sources": [
    {"doc_id": "...", "page": 22, "chunk_id": "abc123"}
  ],
  "relations": [
    {"type": "used_in", "target": "Web authentication"},
    {"type": "contrasts_with", "target": "Implicit Flow"}
  ]
}
```

Memory promotion

Only promote stable, reusable knowledge into long-term agent memory:

- project rules
- recurring preferences
- stable definitions
- course/module summaries
- validated workflows
- system architecture decisions

Do not promote:

- unverified OCR
- temporary RSS headlines
- duplicate chunks
- noisy summaries
- low-confidence extraction
- full copyrighted book content

Dynamic update loop

Source update detection

Source	Detection method
RSS	ETag, Last-Modified, URL hash
Web page	canonical URL + content hash
PDF file	file hash + page hash
University folder	file watcher + hash check
Book	usually static, no recurring check
GitHub repo	commit hash
Obsidian note	frontmatter hash

Update flow

```
scheduled check
  -> detect changed source
  -> compare hashes
  -> extract changed pages/articles only
  -> rebuild affected chunks
  -> delete stale vectors for changed chunks
  -> insert new vectors
  -> update FTS index
  -> update Graphify candidate changes
  -> update Obsidian candidate notes
  -> generate report
```

Use systemd timers first. n8n can be added later for visual workflows, approvals, and notifications, but it should not be the core processing engine.

Dashboard and endpoint workflow

Laptops should be endpoints only.

Recommended user flow:

1. Open dashboard from laptop.
2. Drop file, folder, RSS feed, or URL.
3. Select source category if needed.
4. System registers the source and starts routing.
5. Dashboard shows extraction status.
6. Low-confidence pages appear in review queue.
7. User approves or rejects promotion candidates.
8. Approved notes go to Obsidian and Graphify.
9. RAG endpoint can answer with citations.

Dashboard cards:

- Ingestion queue
- OCR queue
- Failed pages
- Low-confidence tables
- Recently indexed sources
- Obsidian export candidates
- Graphify export candidates
- Retrieval test results
- Storage usage
- GPU queue status

Quality gates

Extraction score

Every document should receive:

- text score
- layout score
- table score
- figure score
- citation/provenance score
- OCR confidence
- language confidence
- final pass/fail status

Required reports

For every batch:

```
extraction_report.md
low_confidence_pages.md
failed_tables.json
changed_sources.json
retrieval_eval.json
promotion_candidates.md
```

Golden test set

Create a small benchmark library:

- 10 digital PDFs
- 10 scanned PDFs
- 5 table-heavy documents
- 5 Romanian documents
- 5 English documents
- 5 mixed Romanian/English documents
- 5 university PDFs
- 5 pages with diagrams or screenshots
- 5 RSS/web sources

Manual checks:

- page count correct
- text not missing
- reading order correct
- table rows and columns preserved
- figures detected
- formulas acceptable
- search returns correct source in top 5
- answer cites correct page

Security and privacy

Treat all documents as untrusted input.

Rules:

- process files in containers where possible
- disable macro execution
- do not auto-run embedded scripts
- preserve raw files as read-only after registration
- keep private documents local
- keep logs free of sensitive full text where possible
- classify sources as private, course, project, public, or unknown
- never send private documents to cloud services without explicit approval
- keep backups encrypted if they include personal or university material

Implementation roadmap

Phase 1: Foundation

Goal: controlled ingestion with no memory writes.

Build:

- folder structure
- SQLite registry
- file hashing
- source registration API
- upload folder
- basic dashboard status
- job state machine
- reports folder

Exit criteria:

- files can be added
- duplicates are detected
- jobs are tracked
- raw files remain immutable

Phase 2: Document routing and extraction

Build:

- document router
- digital PDF parser
- Markdown/JSON output
- page image rendering
- table extraction attempt
- extraction report

Exit criteria:

- digital PDFs produce Markdown and JSON
- every page has page-level metadata
- failed pages are visible

Phase 3: OCR and visual validation

Build:

- OCR fallback
- low-confidence page detection
- OCR/VLM GPU lock
- table repair queue
- scanned PDF path

Exit criteria:

- scanned PDFs can be processed
- low-confidence pages are routed correctly
- OCR does not block normal retrieval

Phase 4: Indexing and RAG

Build:

- chunker
- BGE-M3 embedding worker
- Qdrant collection
- SQLite FTS5 index
- hybrid retrieval endpoint
- answer endpoint with source citations

Exit criteria:

- user can ask questions
- answers cite document/page/chunk
- weak retrieval is reported honestly

Phase 5: Obsidian and Graphify export

Build:

- Obsidian note templates
- Graphify node/edge JSON export
- promotion queue
- review UI

Exit criteria:

- no automatic brain dumping
- reviewed notes can be exported
- graph relationships cite sources

Phase 6: Dynamic updates

Build:

- RSS checker
- folder watcher
- URL checker
- changed-page detector
- incremental re-index
- update reports

Exit criteria:

- only changed material is reprocessed
- stale vectors are removed
- reports show what changed

Phase 7: Evaluation harness

Build:

- golden document set
- extraction scoring
- retrieval scoring
- model comparison reports
- regression tests

Exit criteria:

- changes to parsers/models can be measured
- broken extraction is detected before promotion

Recommended first build stack

Start with:

```
Python
SQLite
PyMuPDF
pdfplumber
Docling
Marker
MinerU
PaddleOCR / PaddleOCR-VL
olmOCR
BGE-M3 via FlagEmbedding
Qdrant
SQLite FTS5
Ollama for local LLM control
systemd timers
Obsidian Markdown export
Graphify JSON export
```

Add later:

```
n8n for visual approvals and notifications
Qwen3 Embedding / reranker after baseline retrieval works
Qwen-VL page validation after OCR routing is stable
Telegram / WhatsApp notifications
```

Working strategy for the AI-PC

Recommended operating mode:

- one GPU-heavy job at a time
- background OCR mainly overnight
- embedding batches after extraction
- live RAG/chat gets priority
- keep batch size small until memory use is measured
- never process a large book without batch checkpoints
- write every intermediate artefact to disk
- keep all hashes and model versions

Good default policy:

```
Small files: process immediately
Books: process in batches
Scanned files: queue for OCR window
RSS: process on schedule
Web pages: process with hash checks
University material: process immediately, then create review candidates
```

Final target

The final system should let you drop in a PDF, scanned document, RSS feed, book, image, or university file and have the AI-PC automatically:

1. register it
2. classify it
3. extract it
4. validate it
5. OCR only where needed
6. create Markdown and JSON
7. chunk it with provenance
8. embed it
9. index it for hybrid retrieval
10. create reviewable Obsidian/Graphify candidates
11. update changed sources later
12. answer questions with citations

The core idea is simple:

> Build a document operating system first, then build RAG on top of it.

A vector database is only one part of the stack. The valuable part is the controlled pipeline that turns messy human-readable material into trustworthy, source-linked knowledge.

Sources checked

Checked on 2026-07-08. Use these as starting references, not as permanent truth. Re-check versions before installing.

- **Docling** - Document parsing and conversion, including PDF understanding, OCR, tables, formulas, and reading order.
  https://www.docling.ai/
- **Marker** - Converts PDF, images, Office files, EPUB, HTML to Markdown, JSON, chunks, and HTML.
  https://github.com/datalab-to/marker
- **MinerU** - Document parsing for PDF, image, DOCX, PPTX, XLSX to Markdown and JSON, useful for scientific/technical documents.
  https://github.com/opendatalab/MinerU
- **PaddleOCR / PaddleOCR-VL** - OCR and document parsing toolkit for PDFs/images, including multilingual document parsing.
  https://github.com/PaddlePaddle/PaddleOCR
- **olmOCR** - Toolkit for converting PDFs and image-based documents into clean text in natural reading order.
  https://github.com/allenai/olmocr

- **Qdrant hybrid queries** - Hybrid retrieval combining dense and sparse representations. https://qdrant.tech/documentation/search/hybrid-queries/
- **SQLite FTS5** - SQLite full-text search virtual table module. https://www.sqlite.org/fts5.html
- **FlagEmbedding / BGE-M3** - BGE-M3 multilingual dense, sparse, and multi-vector retrieval. https://github.com/FlagOpen/FlagEmbedding
- **Qwen3 Embedding** - Embedding and reranking model family for multilingual retrieval. https://github.com/QwenLM/Qwen3-Embedding
- **Ollama Qwen3 14B** - Qwen3 14B local model packaging information for Ollama. https://ollama.com/library/qwen3:14b
- **systemd timers** - Timer-based activation for recurring local jobs. https://man.archlinux.org/man/systemd.timer.5.en

Document operating system for RAG

Baza dinamica locala de cunostinte pentru documente, RAG, Obsidian, Graphify si memoria agentilor

2026-07-08

Contents

Rezumat executiv

Acest plan descrie un **Document operating system for RAG**, adica un sistem local-first pentru ingestie, extragere, validare, indexare si promovare controlata a documentelor in RAG, Obsidian, Graphify si memoria agentilor.

Scopul nu este doar sa incarci fisiere intr-o baza vectoriala. O baza vectoriala este doar un strat. Tinta corecta este o conducta controlata care pastreaza separat sursa bruta, textul extras, Markdown, JSON, imaginile paginilor, tabelele, figurile, chunk-urile, embedding-urile, indexul full-text, nodurile de graph, notitele Obsidian si deciziile de promovare in memorie.

Fluxul dorit este:

```
material sursa
  -> registru de surse
  -> router de documente
  -> conducta de extractie
  -> OCR / validare vision doar unde este nevoie
  -> scoruri de calitate
  -> Markdown si JSON structurat
  -> chunking cu provenienta pe pagina
  -> index de cautare hibrida
  -> serviciu RAG cu citari
```

```
-> promovare revizuita in Obsidian / Graphify / memorie
-> loop de update pentru surse schimbatoare
```

Regula principala este: **nu pierde niciodata provenienta**. Orice raspuns dat din KB trebuie sa poata fi urmarit inapoi la fisierul sursa, pagina, chunk, metoda de extractie, timestamp si scor de incredere.

Presupuneri hardware corectate

Masina activa de procesare este un singur AI-PC cu:

- un singur GPU activ: **RTX 5060 Ti 16 GB**
- CPU: **Intel i7-10750 class**
- RAM: **32 GB DDR4**
- laptopurile sunt doar endpoint-uri pentru vizualizare, upload, verificare si control

Nu proiecta sistemul pentru GPU-uri nefolosite. Nu presupune multi-GPU scheduling. Sistemul trebuie sa ruleze cu o singura coada pentru GPU si o coada separata pentru extractie CPU/disk.

Specificatie hardware minima

Aceasta este specificatia minima practica pentru prima versiune folosibila:

- CPU 6-core / 12-thread sau mai bun
- 32 GB RAM
- GPU NVIDIA cu 12 GB VRAM sau mai mult
- SSD 1 TB pentru artefacte active
- stocare separata mare pentru documentele brute si rezultatele generate
- Linux host cu Docker, Python si drivere NVIDIA stabile

AI-PC-ul actual intra in aceasta clasa, mai ales datorita placii cu 16 GB VRAM. Cei 32 GB RAM sunt suficienti pentru inceput, dar batch-urile trebuie controlate.

Specificatie hardware recomandata

Aceasta este tinta recomandata daca sistemul creste:

- CPU cu 8-12 core-uri sau mai bun
- 64 GB RAM
- GPU NVIDIA cu 16 GB VRAM sau mai mult
- NVMe 2 TB pentru procesare activa
- 4 TB+ SSD/HDD pentru biblioteca bruta, imagini de pagina si arhive de artefacte
- tinta de backup zilnic
- UPS daca ruleaza joburi lungi de OCR/indexare

Primul upgrade logic ar fi **RAM la 64 GB**, nu inca un GPU. PDF-urile mari, randarea paginilor, OCR-ul, batch-urile de embedding, Qdrant si dashboard-ul pot presa 32 GB inainte sa fie saturat GPU-ul.

Principii de design

1. Router intai, OCR dupa

Nu face OCR pe fiecare PDF. Este lent si poate strica precizia atunci cand textul embedded este deja bun.

Fiecare fisier trebuie sa treaca intai printr-un router de documente care decide:

- este PDF digital?
- este PDF scanat?
- contine tabele?
- contine formule?
- este multi-coloana?
- contine grafice, figuri sau screenshot-uri?

- este romana, engleza sau mixt?
- este material de universitate, carte, articol, RSS sau captura web?
- are nevoie de fallback?

Apoi sistemul alege cea mai ieftina metoda de extractie care ramane fiabila.

2. Stocheaza straturi, nu doar chunk-uri

Sistemul nu trebuie sa pastreze doar chunk-uri vectoriale. Trebuie sa pastreze:

- fisierul brut
- metadatele sursei
- textul extras
- Markdown extras
- imaginile paginilor
- output OCR
- layout JSON
- tabele JSON / CSV
- descrieri de figuri
- chunk-uri
- embedding-uri
- randuri full-text search
- noduri si muchii de graph
- notite Obsidian
- decizii de promovare in memorie

Vectorii sunt regenerabili. Sursele brute si artefactele de extractie nu sunt regenerabile fara cost si risc.

3. Cautare hibrida, nu doar vector search

Vector search este bun pentru similaritate semantica, dar este slab pentru fapte exacte, coduri de module, formulare juridica, comenzi, referinte de pagina, etichete de tabel si termeni tehnici rari.

Foloseste cautare hibrida:

```
query
  -> keyword / BM25 / SQLite FTS
  -> dense vector search
  -> optional sparse vector search
  -> filtre metadata
  -> reranker
  -> raspuns cu citari de sursa
```

4. Promovarea in memorie trebuie controlata

Nu impinge automat fiecare propozitie in Obsidian, Graphify sau memoria lunga a agentului.

Foloseste niveluri de promovare:

- L0 evidenta bruta
- L1 artefacte extrase din document
- L2 chunk-uri indexate
- L3 notite Obsidian curate
- L4 concepte si relatii Graphify
- L5 memorie stabila pentru agent

Doar L3-L5 trebuie tratate ca knowledge pe termen lung. Restul sunt materiale de evidenta si retrieval.

5. Precizia bate viteza automatizarii

Sistemul trebuie sa prefere procesare mai lenta, dar corecta, fata de extractie rapida si rupta. Fiecare pass de extractie trebuie sa produca scor de incredere si raport de esec.

Arhitectura tinta

Servicii principale

Foloseste un numar mic de servicii clare:

- **ingest-api**: primeste fisiere, URL-uri, feed-uri RSS si joburi manuale
- **registry-db**: SQLite pentru surse, documente, pagini, joburi, chunk-uri si scoruri
- **worker-extract**: parser CPU pentru documente digitale
- **worker-ocr-vlm**: worker OCR/vision blocat pe GPU
- **worker-embed**: worker pentru embedding-uri
- **qdrant**: baza vectoriala
- **fts-index**: index full-text SQLite FTS5
- **rag-api**: serviciu de retrieval si raspuns
- **exporter-obsidian**: generator de notite Markdown
- **exporter-graphify**: generator de noduri si relatii
- **review-ui**: coada pentru pagini slabe si promovari
- **scheduler**: systemd timers pentru RSS, fisiere schimbate si reindexare

Structura de foldere

```
/srv/ai-workdesk/kb/
  raw/
    books/
    magazines/
    university/
    rss/
    web/
    images/
  registry/
    kb.sqlite
  artefacts/
    extracted_text/
    extracted_markdown/
    page_images/
    ocr_json/
    layout_json/
    tables/
    figures/
    chunks/
    reports/
  indexes/
    qdrant/
    sqlite_fts/
  exports/
    obsidian/
    graphify/
    memory_candidates/
  eval/
    golden_docs/
    extraction_scores/
    retrieval_scores/
```

State machine pentru joburi

Fiecare sursa trebuie sa treaca prin stari explicite:

```
REGISTERED
  -> ROUTED
  -> EXTRACTED
  -> VALIDATED
  -> CHUNKED
```

```
  -> EMBEDDED
  -> INDEXED
  -> REVIEWED
  -> PROMOTED
  -> AVAILABLE_FOR_RAG
```

Joburile esuate nu trebuie sa dispara. Ele trebuie mutate in:

```
NEEDS_RETRY
NEEDS_SECONDARY_OCR
NEEDS_MANUAL_REVIEW
BLOCKED_UNSUPPORTED_FORMAT
```

Registrul de surse

Registrul este panoul de control. El previne munca duplicata si permite update-uri fiabile.

Tabele recomandate:

sources

```
source_id
source_kind: file | rss | url | folder_watch | manual_note
original_location
canonical_location
sha256
size_bytes
created_at
registered_at
last_checked_at
licence_status
privacy_class
status
```

documents

```
doc_id
source_id
title
author
language
content_type
page_count
parser_selected
ocr_required
created_at
updated_at
```

pages

```
page_id
doc_id
page_number
page_hash_text
page_hash_image
text_confidence
layout_confidence
table_confidence
ocr_used
needs_review
```

chunks

```
chunk_id
doc_id
page_start
page_end
section_title
chunk_type
text
summary
keywords
entities
embedding_model
source_hash
created_at
```

jobs

```
job_id
source_id
doc_id
job_type
state
priority
started_at
finished_at
error_message
retry_count
```

Rutarea documentelor

Routerul trebuie sa aleaga strategia de extractie inainte de procesare scumpa.

Reguli de rutare

Conditie	Prima actiune	Fallback
PDF digital cu text bun	PyMuPDF / pdfplumber / Docling	Marker sau MinerU
PDF scanat	pipeline OCR	validare VLM pe pagini
PDF tehnic cu formule	MinerU / Marker	validare VLM pe pagini esuate
Revista sau layout complex	Docling / Marker	OCR/VLM pe pagini cu scor mic
Imagine sau screenshot	OCR + descriere VLM	review manual daca scorul e mic
Articol RSS	feed parser + extractie continut	browser snapshot daca pagina e dinamica
Pagina web	trafilatura/readability/browser capture	screenshot + OCR daca este blocata
DOCX/PPTX/XLSX	Docling / Marker / MinerU	conversie LibreOffice ca fallback

Output router

```
{
  "doc_id": "...",
  "route": "digital_pdf_structured",
  "language": "en_ro_mixed",
  "parser_chain": ["docling", "marker_fallback"],
  "ocr_policy": "selective",
```

```
  "vlm_policy": "low_confidence_pages_only",
  "expected_risk": "tables_and_two_column_layout"
}
```

Conducta de extractie

Pass 1: extractie normala

Pentru PDF-uri digitale, incepe cu extractia normala:

- text embedded
- headings
- numere de pagina
- tabele
- figuri
- captions
- linkuri
- referinte
- reading order
- formule unde este posibil

Unelte candidate:

- PyMuPDF pentru text rapid si metadata de pagina
- pdfplumber pentru inspectie text si tabele
- Docling pentru conversie structurata de documente
- Marker pentru Markdown/JSON/chunks
- MinerU pentru documente tehnice si stiintifice

Pass 2: OCR si vision selectiv

Ruleaza OCR doar unde este nevoie:

- pagina nu are text embedded
- textul embedded este corupt
- textul romanesc sau englezesc este rupt
- scorul tabelului este mic
- extractia formulelor este slaba
- pagina are screenshot-uri, diagrame, grafice sau scris de mana
- reading order pare gresit

Unelte/modele candidate:

- PaddleOCR / PaddleOCR-VL pentru OCR multilingv si parsing de documente
- olmOCR pentru documente PDF/imagine unde reading order conteaza
- Qwen2.5-VL sau Qwen3-VL class model pentru validare pe pagini selectate

Pentru ca AI-PC-ul are un singur GPU de 16 GB, toate joburile OCR/VLM trebuie sa foloseasca un **GPU lock**. Un singur job GPU greu trebuie sa ruleze la un moment dat.

Pass 3: validarea extractiei

Fiecare pagina primeste scor:

```
{
  "page_number": 12,
  "text_confidence": 0.94,
  "layout_confidence": 0.88,
  "table_confidence": 0.72,
  "ocr_used": true,
  "needs_review": false,
  "warnings": ["possible_table_loss"]
}
```

Verificari:

- pagina goala sau aproape goala
- eliminare header/footer repetat
- reading order gresit
- cuvinte rupte intre randuri
- sanity check pentru randuri/coloane de tabel
- verificare formule
- detectare imagine/figura
- mismatch de limba
- pagini duplicate

Strategia OCR si VLM pe un singur GPU

GPU-ul de 16 GB este suficient pentru o conducta locala serioasa, dar numai daca joburile sunt puse in coada.

Regula recomandata:

```
Extractia CPU poate rula in paralel.
OCR/VLM pe GPU ruleaza un singur job la un moment dat.
Batch-urile de embedding ruleaza cand OCR/VLM este idle.
Modelul de chat/retrieval primeste prioritate peste procesarea de fundal.
```

Scheduling practic

Foloseste trei cozi:

- **fast queue**: PDF-uri mici digitale, RSS, pagini web
- **batch queue**: carti, reviste, materiale mari de universitate
- **heavy queue**: PDF-uri scanate, OCR, validare VLM, reparare tabele

Politica exemplu:

```
08:00-22:00: ingestie usoara si prioritate retrieval
22:00-07:00: batch OCR, embedding-uri, rebuild de index
```

Daca sistemul este folosit pentru studiu sau coding, opreste OCR-ul greu.

Strategie de modele si unelte

Rationament local si control agentic

Candidate recomandate:

- Qwen3 14B quantized prin Ollama pentru rationament local principal
- Qwen3 8B pentru clasificare si rutare mai rapida
- Qwen2.5-Coder 14B sau similar pentru reparatii de cod si scripturi de pipeline

Foloseste LLM-ul pentru decizii, nu pentru extractie bruta. Parser-ele si OCR-ul trebuie sa extraga. LLM-ul trebuie sa aleaga ruta, sa valideze sumarizari, sa creeze metadata si sa produca candidati de memorie.

Embedding-uri

Recomandare principala:

- **BGE-M3** pentru retrieval multilingv, mai ales material mixt engleza/romana

De ce:

- multilingv
- dense retrieval
- sparse retrieval
- multi-vector retrieval
- suporta granularitate mai lunga decat multe modele mici de embedding

Candidat viitor:

- **Qwen3 Embedding 0.6B / 4B / 8B**, testat dupa ce conducta de baza merge

Nu schimba modelele de embedding la intamplare. Stocheaza numele si versiunea modelului de embedding pentru fiecare chunk.

Reranking

Adauga reranking dupa ce retrieval-ul hibrid de baza merge.

Candidati:

- familia BGE reranker
- familia Qwen3 reranker

Reranking-ul trebuie sa fie optional pentru ca adauga latenta. Foloseste-l pentru raspunsuri finale, nu pentru fiecare cautare exploratorie.

Modele vision/document

Candidate:

- PaddleOCR / PaddleOCR-VL pentru OCR si document parsing multilingv
- olmOCR pentru documente PDF/imagine unde reading order conteaza
- Qwen2.5-VL sau Qwen3-VL class model pentru review vizual al paginilor cu scor mic

Nu pune un VLM sa proceseze carti intregi pagina cu pagina daca parser-ul normal nu a esuat. VLM-urile sunt mai scumpe si mai lente.

Strategie de chunking

Chunking-ul trebuie sa pastreze sensul si evidenta pe pagina.

Tipuri de chunk

Foloseste tipuri diferite:

- definitie
- explicatie
- procedura
- tabel
- figura
- cod
- citat
- formula
- sumar
- referinta
- intrebare-raspuns
- nota de curs / assignment

Reguli de marime

Tip sursa	Regula de chunking
Note de universitate	pe heading, 400-900 cuvinte
Carti	pe sectiune, 700-1200 cuvinte
Reviste	pe articol/sectiune
Tabele	un tabel = un chunk plus sumar
Cod/comenzi	pastreaza intact
Diagrame	text OCR + caption + imagine pagina legata
RSS/web	sectiuni de articol cu URL canonic
Definitii	chunk-uri atomice separate

Structura parent-child

Foloseste un arbore:

```
document
  chapter
    section
      chunk
        table
        figure
        quote
```

Astfel RAG-ul poate gasi un chunk mic, dar poate extinde contextul la sectiunea parinte cand este nevoie.

Retrieval si serviciu de raspuns

Pipeline de retrieval

```
query utilizator
  -> detectare limba
  -> normalizare / query expansion
  -> filtre metadata
  -> SQLite FTS / BM25 search
  -> dense vector search
  -> optional sparse vector search
  -> fusion rezultate
  -> reranking
  -> context pack
  -> raspuns cu citari
```

Filtre metadata

Suporta filtre precum:

- tip sursa: universitate, carte, RSS, web, imagine
- cod modul
- autor
- an
- limba
- prag de incredere
- data ingestiei
- doar reviewed
- include/exclude pagini doar OCR
- clasificare private/public

Regula de raspuns

Serviciul de raspuns nu trebuie sa spuna ca ceva vine din KB daca nu poate cita sursa.

Fiecare raspuns generat trebuie sa includa:

```
Surse:
- titlu document
- numar pagina sau URL articol
- chunk ID
- confidence de extractie
```

Daca retrieval-ul este slab, raspunsul trebuie sa spuna:

```
Am gasit material apropiat, dar nu pot confirma raspunsul din sursele indexate.
```

Export Obsidian si Graphify

Export Obsidian

Obsidian trebuie sa primeasca notite curate, nu dump brut.

Tipuri de notite recomandate:

- notite de sursa
- notite de concept
- notite de modul
- harti de topic
- definitii
- suport pentru assignment
- sumare saptamanale de universitate
- sumare de lectura
- intrebari nerezolvate

Exemplu frontmatter:

```yaml
title: OAuth 2.0 Authorization Code Flow
type: concept
source_count: 3
confidence: reviewed
created: 2026-07-08
updated: 2026-07-08
tags:
  - cybersecurity
  - authentication
  - university
```

Exemplu corp de nota:

```
# OAuth 2.0 Authorization Code Flow

## Summary
Sumar scurt pentru om.

## Key points
- Punct 1
- Punct 2

## Source evidence
- COM4010 Lecture 04, page 22, chunk abc123
- Security textbook, page 118, chunk def456

## Related
- [[OpenID Connect]]
- [[Access Token]]
- [[Refresh Token]]
```

Export Graphify

Graphify trebuie sa primeasca entitati si relatii:

```json
{
  "node_type": "concept",
  "name": "OAuth 2.0 Authorization Code Flow",
  "aliases": ["auth code flow"],
  "sources": [
    {"doc_id": "...", "page": 22, "chunk_id": "abc123"}
  ],
  "relations": [
    {"type": "used_in", "target": "Web authentication"},
    {"type": "contrasts_with", "target": "Implicit Flow"}
```

```
  ]
}
```

Promovare in memorie

Promoveaza in memoria lunga a agentului doar cunostinte stabile si reutilizabile:

- reguli de proiect
- preferinte recurente
- definitii stabile
- sumare de curs/modul
- workflow-uri validate
- decizii de arhitectura

Nu promova:

- OCR neverificat
- headline-uri RSS temporare
- chunk-uri duplicate
- sumare zgomotoase
- extractie cu scor mic
- continut complet din carti protejate de copyright

Loop de update dinamic

Detectare update sursa

Sursa	Metoda de detectare
RSS	ETag, Last-Modified, URL hash
Pagina web	URL canonic + content hash
PDF fisier	file hash + page hash
Folder universitate	file watcher + hash check
Carte	de obicei static, fara check recurent
GitHub repo	commit hash
Nota Obsidian	frontmatter hash

Flow de update

```
check programat
  -> detecteaza sursa schimbata
  -> compara hash-uri
  -> extrage doar pagini/articole schimbate
  -> reconstruieste chunk-urile afectate
  -> sterge vectorii vechi pentru chunk-uri schimbate
  -> insereaza vectori noi
  -> actualizeaza indexul FTS
  -> actualizeaza candidatii Graphify
  -> actualizeaza candidatii Obsidian
  -> genereaza raport
```

Foloseste systemd timers la inceput. n8n poate fi adaugat mai tarziu pentru workflow vizual, aprobari si notificari, dar nu ar trebui sa fie motorul principal de procesare.

Dashboard si workflow pe endpoint-uri

Laptopurile trebuie sa fie doar endpoint-uri.

Workflow recomandat:

1. Deschizi dashboard-ul de pe laptop.

2. Arunci fisier, folder, RSS feed sau URL.
3. Alegi categoria sursei daca este nevoie.
4. Sistemul inregistreaza sursa si porneste rutarea.
5. Dashboard-ul arata statusul de extractie.
6. Paginile cu scor mic apar in coada de review.
7. Aprobi sau respingi candidatii de promovare.
8. Notitele aprobate merg in Obsidian si Graphify.
9. Endpoint-ul RAG raspunde cu citari.

Carduri utile in dashboard:

- coada de ingestie
- coada OCR
- pagini esuate
- tabele cu scor mic
- surse indexate recent
- candidati export Obsidian
- candidati export Graphify
- rezultate teste retrieval
- utilizare stocare
- status coada GPU

Quality gates

Scor de extractie

Fiecare document trebuie sa primeasca:

- scor text
- scor layout
- scor tabel
- scor figura
- scor citare/provenienta
- confidence OCR
- confidence limba
- status final pass/fail

Rapoarte necesare

Pentru fiecare batch:

```
extraction_report.md
low_confidence_pages.md
failed_tables.json
changed_sources.json
retrieval_eval.json
promotion_candidates.md
```

Golden test set

Creeaza o biblioteca mica de benchmark:

- 10 PDF-uri digitale
- 10 PDF-uri scanate
- 5 documente cu multe tabele
- 5 documente in romana
- 5 documente in engleza
- 5 documente mixte romana/engleza
- 5 PDF-uri de universitate
- 5 pagini cu diagrame sau screenshot-uri
- 5 surse RSS/web

Verificari manuale:

- numar pagini corect
- textul nu lipseste
- reading order corect
- randuri si coloane de tabel pastrate
- figuri detectate
- formule acceptabile
- cautarea returneaza sursa corecta in top 5
- raspunsul citeaza pagina corecta

Securitate si confidentialitate

Trateaza toate documentele ca input nevalidat.

Reguli:

- proceseaza fisiere in containere unde este posibil
- dezactiveaza executia de macro-uri
- nu rula automat scripturi embedded
- pastreaza fisierele brute read-only dupa inregistrare
- tine documentele private local
- evita loguri cu text complet sensibil
- clasifica sursele ca private, course, project, public sau unknown
- nu trimite documente private in cloud fara aprobare explicita
- tine backup-urile criptate daca includ materiale personale sau universitare

Roadmap de implementare

Faza 1: Fundatie

Scop: ingestie controlata fara scrieri in memorie.

Construieste:

- structura de foldere
- registru SQLite
- hashing fisiere
- API de inregistrare sursa
- folder de upload
- status simplu in dashboard
- job state machine
- folder de rapoarte

Criterii de iesire:

- fisierele pot fi adaugate
- duplicatele sunt detectate
- joburile sunt urmarite
- fisierele brute raman imutabile

Faza 2: Rutare si extractie documente

Construieste:

- router de documente
- parser PDF digital
- output Markdown/JSON
- randare imagine pagina
- incercare extractie tabele
- raport de extractie

Criterii de iesire:

- PDF-urile digitale produc Markdown si JSON
- fiecare pagina are metadata

- paginile esuate sunt vizibile

Faza 3: OCR si validare vizuala

Construieste:

- fallback OCR
- detectie pagini cu scor mic
- OCR/VLM GPU lock
- coada reparare tabele
- ruta pentru PDF scanat

Criterii de iesire:

- PDF-urile scanate pot fi procesate
- paginile slabe sunt rutate corect
- OCR-ul nu blocheaza retrieval-ul normal

Faza 4: Indexare si RAG

Construieste:

- chunker
- worker embedding BGE-M3
- colectie Qdrant
- index SQLite FTS5
- endpoint retrieval hibrid
- endpoint raspuns cu citari

Criterii de iesire:

- utilizatorul poate pune intrebari
- raspunsurile citeaza document/pagina/chunk
- retrieval-ul slab este raportat onest

Faza 5: Export Obsidian si Graphify

Construieste:

- template-uri Obsidian
- export JSON nod/muchie Graphify
- coada de promovare
- review UI

Criterii de iesire:

- nu exista dump automat in brain
- notitele revizuite pot fi exportate
- relatiile graph citeaza surse

Faza 6: Update-uri dinamice

Construieste:

- RSS checker
- folder watcher
- URL checker
- detector pagini schimbate
- reindexare incrementala
- rapoarte de update

Criterii de iesire:

- doar materialul schimbat este reprocesat
- vectorii vechi sunt eliminati
- rapoartele arata ce s-a schimbat

Faza 7: Evaluation harness

Construieste:

- set de documente golden
- scoruri de extractie
- scoruri de retrieval
- rapoarte comparatie modele
- teste de regresie

Criterii de iesire:

- schimbarile la parsere/modele pot fi masurate
- extractia rupta este detectata inainte de promovare

Stack recomandat pentru primul build

Incepe cu:

```
Python
SQLite
PyMuPDF
pdfplumber
Docling
Marker
MinerU
PaddleOCR / PaddleOCR-VL
olmOCR
BGE-M3 via FlagEmbedding
Qdrant
SQLite FTS5
Ollama pentru control LLM local
systemd timers
export Markdown Obsidian
export JSON Graphify
```

Adauga mai tarziu:

```
n8n pentru aprobari vizuale si notificari
Qwen3 Embedding / reranker dupa ce retrieval-ul de baza merge
Qwen-VL pentru validare de pagina dupa ce ruta OCR este stabila
notificari Telegram / WhatsApp
```

Strategie de lucru pentru AI-PC

Mod recomandat de operare:

- un singur job GPU greu la un moment dat
- OCR de fundal mai ales noaptea
- batch-uri de embedding dupa extractie
- live RAG/chat are prioritate
- batch size mic pana masori memoria
- nu procesa o carte mare fara checkpoint-uri de batch
- scrie fiecare artefact intermediar pe disk
- pastreaza toate hash-urile si versiunile de model

Politica implicita buna:

```
Fisiere mici: proceseaza imediat
Carti: proceseaza in batch-uri
Fisiere scanate: coada pentru fereastra OCR
RSS: proceseaza programat
Pagini web: proceseaza cu hash checks
Materiale de universitate: proceseaza imediat, apoi creeaza candidati de review
```

Tinta finala

Sistemul final trebuie sa iti permita sa arunci un PDF, document scanat, feed RSS, carte, imagine sau material de universitate si AI-PC-ul sa faca automat:

1. inregistrare
2. clasificare
3. extractie
4. validare
5. OCR doar unde este nevoie
6. Markdown si JSON
7. chunking cu provenienta
8. embedding
9. indexare pentru cautare hibrida
10. candidati review pentru Obsidian/Graphify
11. update ulterior pentru surse schimbate
12. raspunsuri cu citari

Ideea de baza este simpla:

> Construieste intai un sistem de operare pentru documente, apoi pune RAG peste el.

Baza vectoriala este doar o piesa din stack. Valoarea reala este conducta controlata care transforma material uman dezordonat in knowledge de incredere, legat de surse.

Surse verificate

Verificate la 2026-07-08. Foloseste aceste surse ca punct de plecare, nu ca adevar permanent. Reverifica versiunile inainte de instalare.

- **Docling** - Document parsing and conversion, including PDF understanding, OCR, tables, formulas, and reading order.
  https://www.docling.ai/
- **Marker** - Converts PDF, images, Office files, EPUB, HTML to Markdown, JSON, chunks, and HTML.
  https://github.com/datalab-to/marker
- **MinerU** - Document parsing for PDF, image, DOCX, PPTX, XLSX to Markdown and JSON, useful for scientific/technical documents.
  https://github.com/opendatalab/MinerU
- **PaddleOCR / PaddleOCR-VL** - OCR and document parsing toolkit for PDFs/images, including multilingual document parsing.
  https://github.com/PaddlePaddle/PaddleOCR
- **olmOCR** - Toolkit for converting PDFs and image-based documents into clean text in natural reading order.
  https://github.com/allenai/olmocr
- **Qdrant hybrid queries** - Hybrid retrieval combining dense and sparse representations.
  https://qdrant.tech/documentation/search/hybrid-queries/
- **SQLite FTS5** - SQLite full-text search virtual table module.
  https://www.sqlite.org/fts5.html
- **FlagEmbedding / BGE-M3** - BGE-M3 multilingual dense, sparse, and multi-vector retrieval.
  https://github.com/FlagOpen/FlagEmbedding
- **Qwen3 Embedding** - Embedding and reranking model family for multilingual retrieval.
  https://github.com/QwenLM/Qwen3-Embedding
- **Ollama Qwen3 14B** - Qwen3 14B local model packaging information for Ollama.
  https://ollama.com/library/qwen3:14b
- **systemd timers** - Timer-based activation for recurring local jobs.
  https://man.archlinux.org/man/systemd.timer.5.en